Simulating Network Routing And VLAN Configuration In Linux

Akintunde Ridwan Oladimeji
Student ID : 2025/INT/11578

Course Name : INT303 Networking Fundamentals

Instructor Name : Mr Aminu Idris (AMCPN)

Date of Submission : 27/1/2026
Version : 1.0

Table of Contents

1.0 Introduction

2.0 Aim and objectives

2.1 Tools

3.0 Methodology

3.1 Creating a dummy interface and Setting Static Route

3.2 VLAN Configuration Using Network Namespaces

3.3 IP Address Assignment & Subnetting / Routing toward OWASP

3.4 Connectivity Testing with Ping and Traceroute

3.5 iptables Firewall Rules

3.6 Monitoring Traffic with tcpdump

4.0 Results and Analysis

5.0 Conclusion and Recommendation

6.0 Challenges and Resolution

Executive Summary

This lab focused on creating and configuration of vlans and routes on a linux system with defined route and gateway for packet flow from source such as vlans to destination such as owasp bwa. Key activities include setting up and configuring VLANs on Linux, setting up routes for predefined traffic navigation, testing connectivity between VLANs and other networks, configuring and testing effectiveness of firewalls rules on Linux, setting and testing traffic monitoring processes on Linux. The lab revealed that VLAN configuration, routing and firewalls on Linux enhanced security and traffic management.

1.0 Introduction

Network routing and VLAN configurations in Linux is the process of controlling how data moves between different networks and how one physical network can be logically divided into multiple virtual networks.

Routing defines the path packets take between subnets using routing tables and gateways, while VLANs (Virtual Local Area Networks) allow traffic separation by creating virtual interfaces with different VLAN IDs on the same physical link. Together, they help organize, secure, and manage network traffic in Linux-based network environments.

2.0 Aim and objectives

The aim of this lab is to simulate network routing and VLAN configuration using Linux-based tools while using the OWASP Broken Web Application VM's IP address to test connectivity and routing principles.

The Objectives of this lab is to

- set up and configure VLANs on Linux.
- Set up routes for predefined traffic navigation
- test connectivity between VLANs and other networks
- Configure and test effectiveness of firewalls rules on Linux
- set and test traffic monitoring processes on Linux
- critically study the operational behavior of intra-network and inter-network connectivity and routing in Linux.

2.1 Tools

- Kali Linux
- OWASP BWA
- Iptables

3.0 Methodology

3.1 Creating a dummy interface and Setting Static Route

Creating a Dummy interface

- **Command** : **`sudo ip link add dummy0 type dummy`**
- **Purpose** : Creates a virtual interface that can be used for routing/testing without using a real network interface.
- **Command** : **`sudo ip addr add 192.168.100.1/24 dev dummy0`**
- **Purpose**: Assigns an IP to dummy0 for routing simulations.
- **Command** : **`sudo ip link set dummy0 up`**
- **Purpose**: Brings the interface up so it can transmit/receive packets.

Add a static route to OWASP VM

Purpose: Tells Kali that packets for OWASP subnet go via dummy0

- **Command** : **`sudo ip route add 192.168.56.0/24 via 192.168.100.1 dev dummy0`**

Target check: Ping OWASP

Purpose: Verify connectivity from Kali to OWASP VM.

- **Command** : **`ping -c 4 192.168.56.101`**

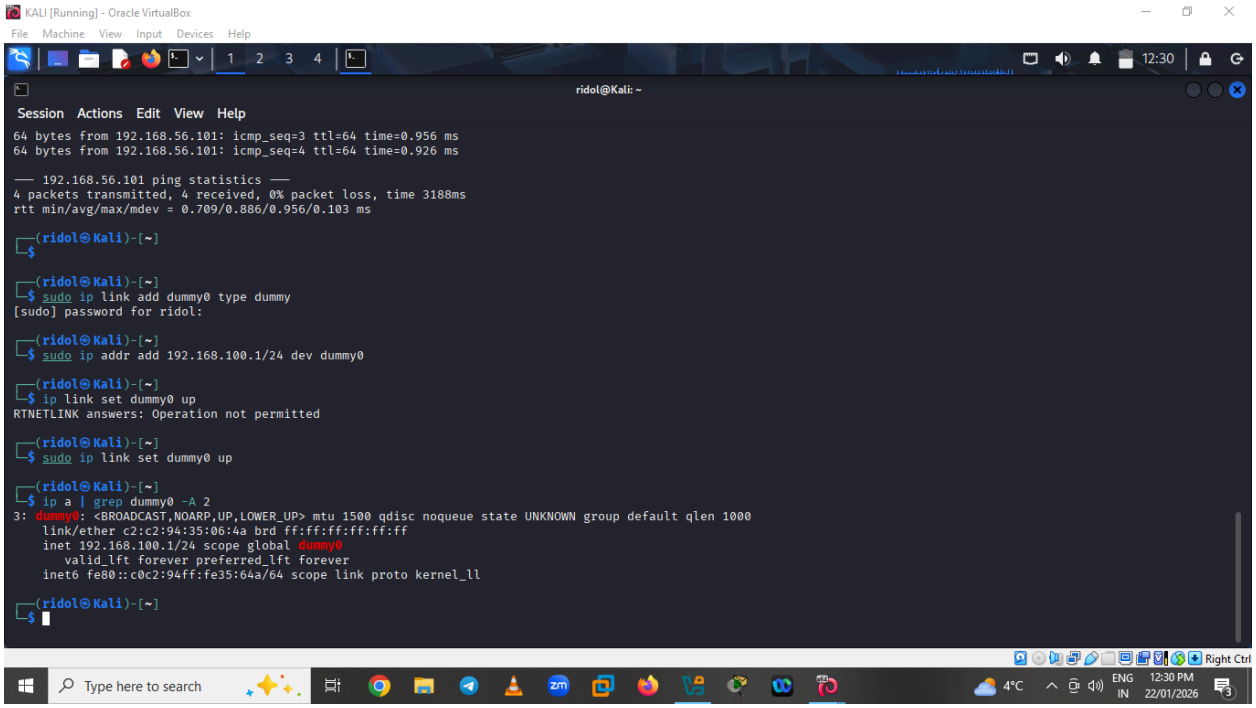

Fig 1:Evidence showing the creation of a dummy interface on Linux terminal

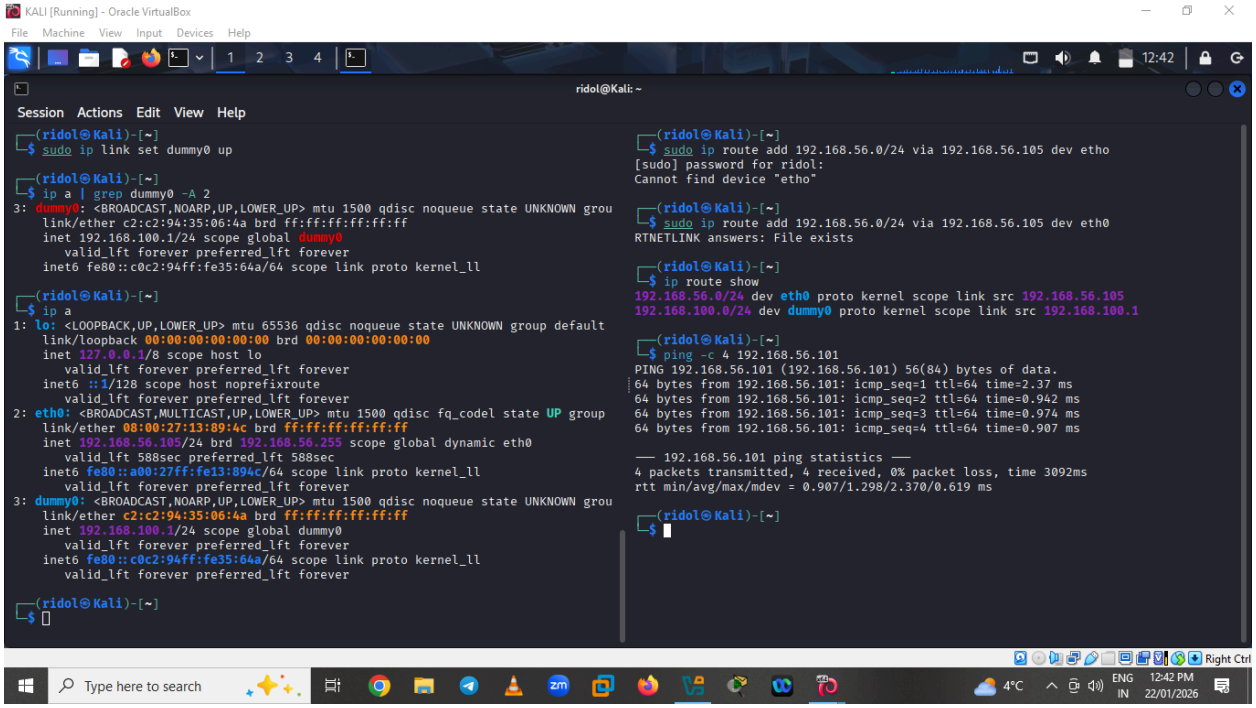

Fig 2: Evidence showing the configuration of static route on Linux terminal

3.2 VLAN Configuration Using Network Namespaces

Create two VLAN namespaces:

Purpose: created two separate VLANs. Each namespace acts like a separate network device.

- **Command** : **`sudo ip netns add vlan1`**
- **Command** : **`sudo ip netns add vlan2`**

Create a veth pair (virtual cable) connecting the VLANs / router interfaces:

Purpose: Creates two virtual Ethernet links; one end goes to VLAN (veth1h, veth2h), the other to Kali router (veth1r, veth2r).

- **Command** : **`sudo ip link add veth1h type veth peer name veth1r`**
- **Command** : **`sudo ip link add veth2h type veth peer name veth2r`**

Move VLAN ends into namespaces:

Purpose: Connects VLAN namespaces to the router interfaces.

- **Command** : **`sudo ip link set veth1h netns vlan1`**
- **Command** : **`sudo ip link set veth2h netns vlan2`**

Bring up loopback and interfaces in VLANs:

Purpose: Loopback is required for namespace communication; interfaces must be up to send/receive packets.

- **Command : `sudo ip netns exec vlan1 ip link set lo up`**
- **Command : `sudo ip netns exec vlan1 ip link set veth1h up`**
- **Command : `sudo ip netns exec vlan2 ip link set lo up`**
- **Command : `sudo ip netns exec vlan2 ip link set veth2h up`**

Assign IP addresses inside VLANs:

Purpose: Each VLAN has its own subnet.

- **Command : `sudo ip netns exec vlan1 ip addr add 192.168.10.1/24 dev veth1h`**
- **Command : `sudo ip netns exec vlan2 ip addr add 192.168.20.1/24 dev veth2h`**

Bring up router interfaces in root namespace:

Purpose: Kali acts as a router between VLANs and other networks.

- **Command : `sudo ip addr add 192.168.10.254/24 dev veth1r`**
- **Command : `sudo ip addr add 192.168.20.254/24 dev veth2r`**
- **Command : `sudo ip link set veth1r up`**
- **Command : `sudo ip link set veth2r up`**

Set default routes in VLANs:

Purpose: VLANs know where to send packets destined for outside their subnet.

- **Command : `sudo ip netns exec vlan1 ip route add default via 192.168.10.254 dev veth1h`**
- **Command : `sudo ip netns exec vlan2 ip route add default via 192.168.20.254 dev veth2h`**

Test inter-VLAN ping:

Purpose: Confirms routing between VLANs works.

- **Command : `sudo ip netns exec vlan1 ping -c 4 192.168.20.1`**
- **Command : `sudo ip netns exec vlan2 ping -c 4 192.168.10.1`**

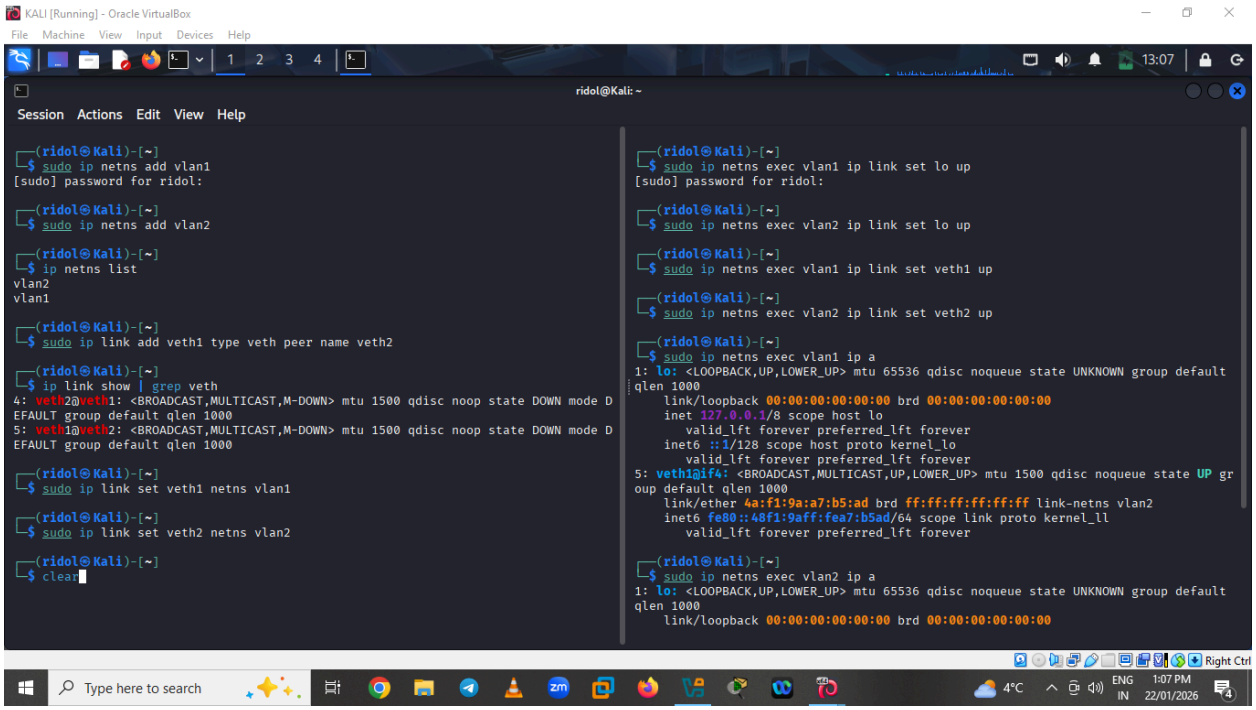

Fig 3: Evidence showing the creation and activation of vlans on Linux terminal

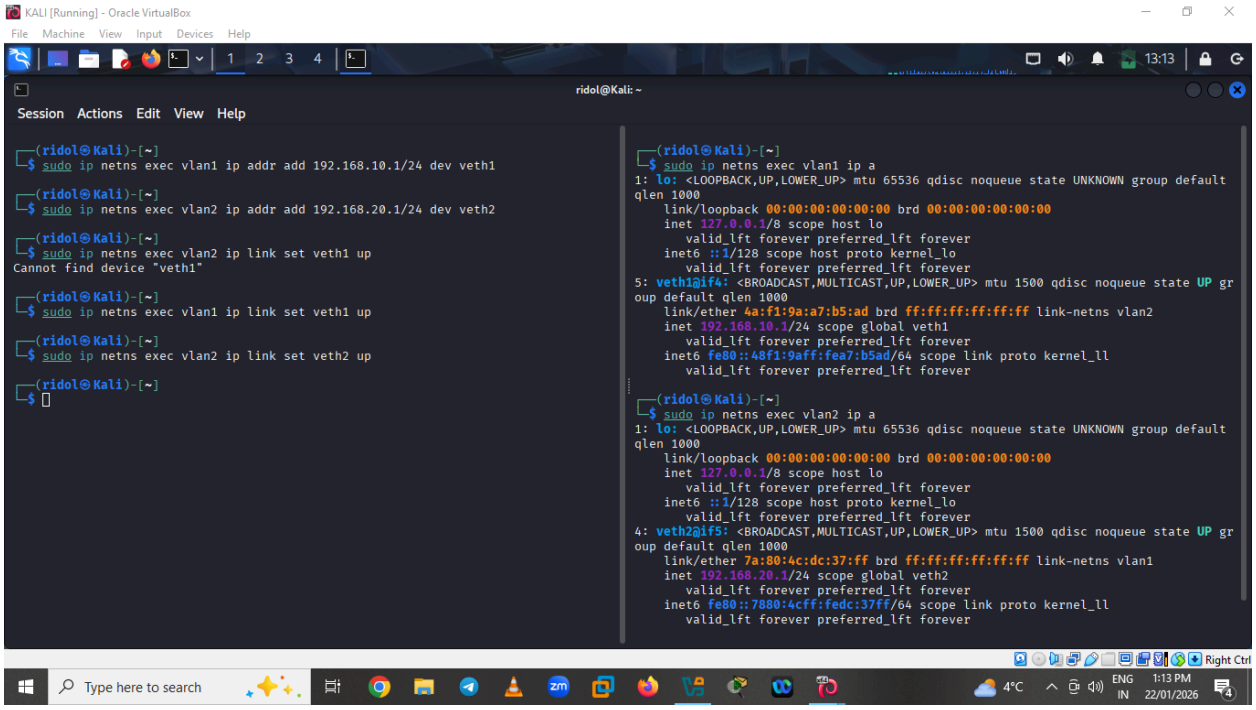

Fig 4: Evidence showing the ip assigned to the vlans on Linux terminal

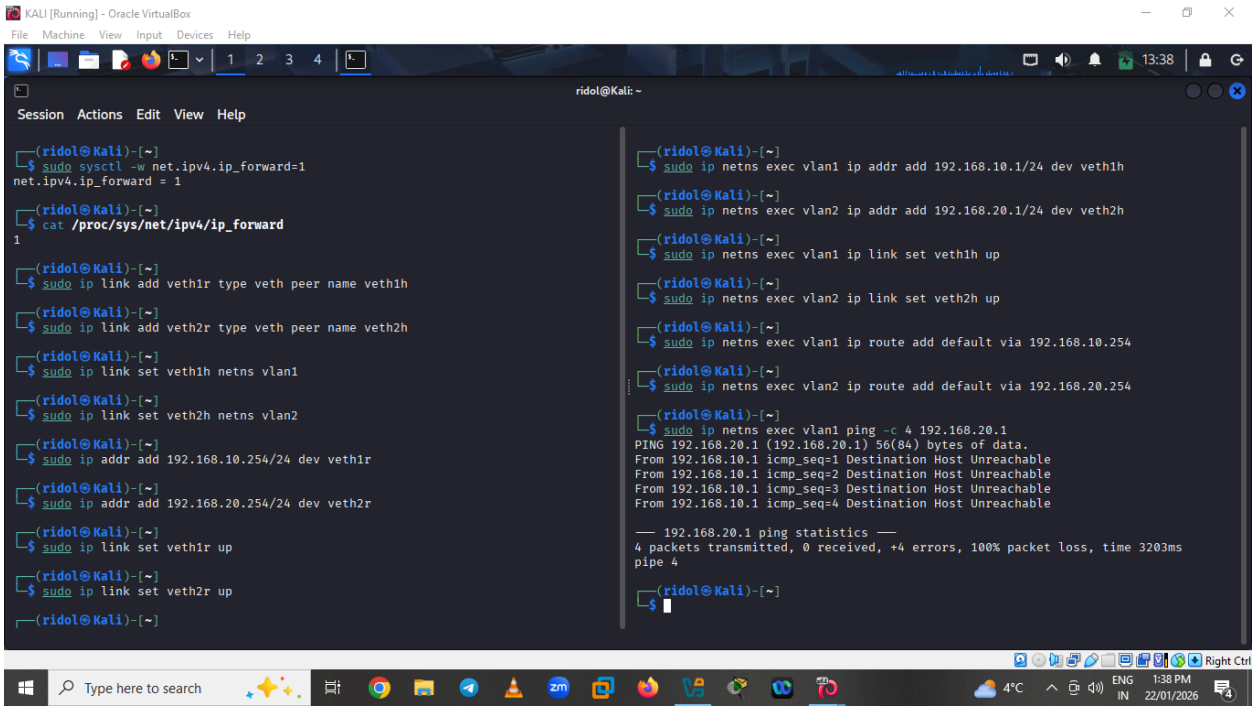

Fig 5: Evidence showing the creation of virtual cable and interfaces for the vlans and router on Linux terminal

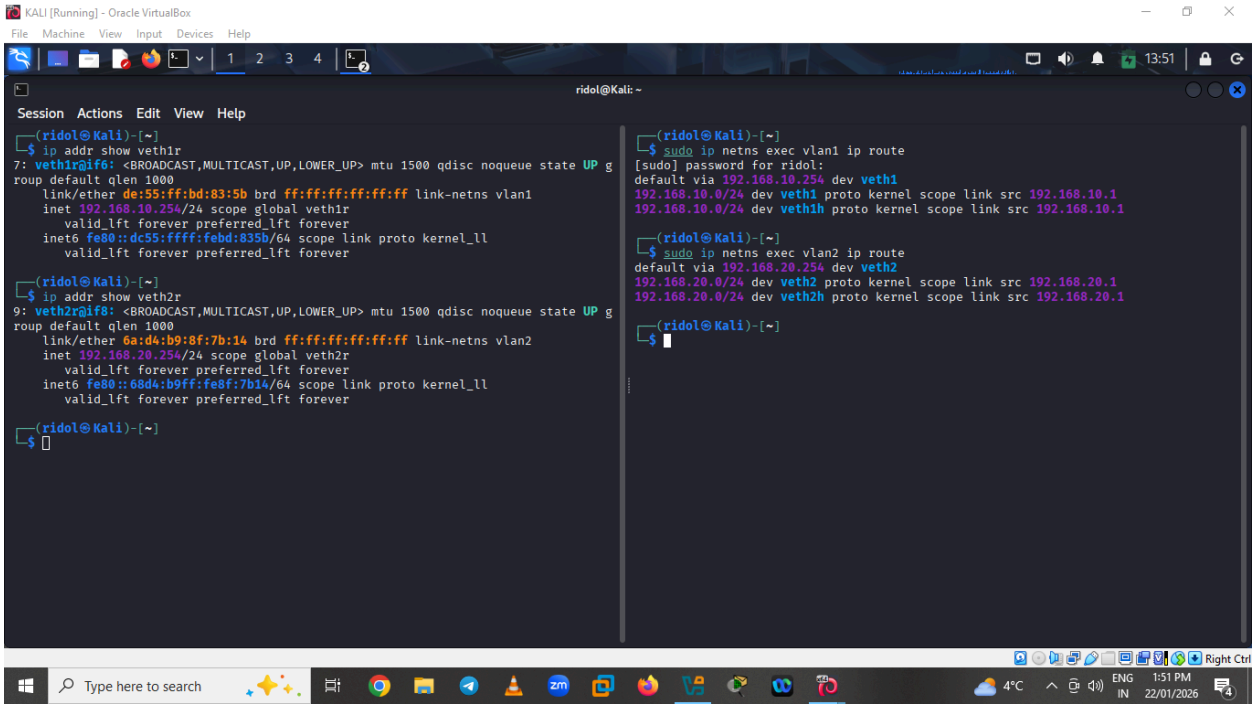

Fig 6: Evidence showing the confirmation of the created interfaces on Linux terminal

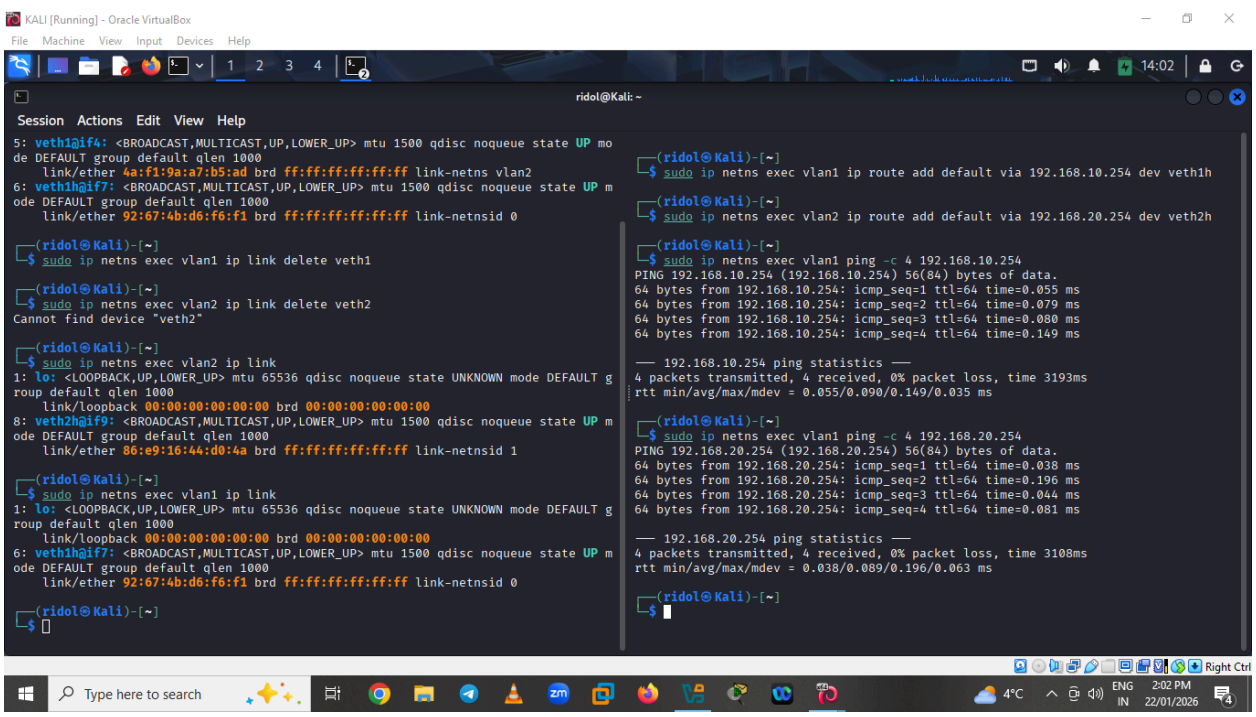

Fig 7: Evidence showing successful ping of the vlans gateways on Linux terminal

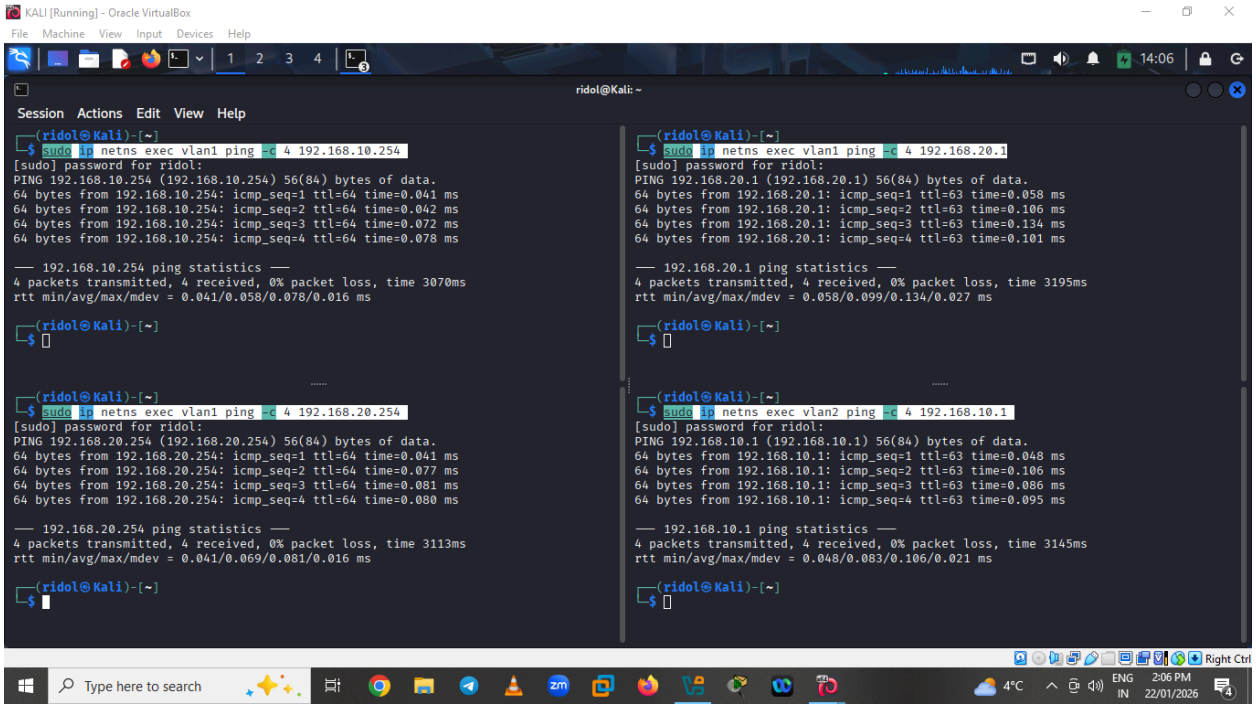

Fig 8: Evidence showing successful inter vlans pings on Linux terminal

3.3 IP Address Assignment & Subnetting / Routing toward OWASP

Add route in Kali to OWASP subnet:

Purpose: Allow VLANs to reach OWASP VM through Kali router. Kali knows how to reach the OWASP VM via its eth0 interface.

- **Command** : **`sudo ip route add 192.168.56.0/24 dev eth0`**

Add routes in VLANs to send OWASP traffic via router

Purpose: Tells VLANs to forward OWASP-bound traffic to Kali.

- **Command** : **`sudo ip netns exec vlan1 ip route add 192.168.56.0/24 via 192.168.10.254`**
- **Command : `sudo ip netns exec vlan2 ip route add 192.168.56.0/24 via 192.168.20.254`**

Add NAT/Masquerade on Kali for VLAN (OWASP traffic)

Purpose: Rewrites source IP so OWASP replies reach Kali then forwarded to VLANs.

- **Command** : `sudo iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -o eth0 -j MASQUERADE`
- **Command** : `sudo iptables -t nat -A POSTROUTING -s 192.168.20.0/24 -o eth0 -j MASQUERADE`

Test ping from VLANs to OWASP:

Purpose: Confirms VLANs can reach OWASP.

- **Command** : `sudo ip netns exec vlan1 ping -c 4 192.168.56.101`
- **Command** : `sudo ip netns exec vlan2 ping -c 4 192.168.56.101`

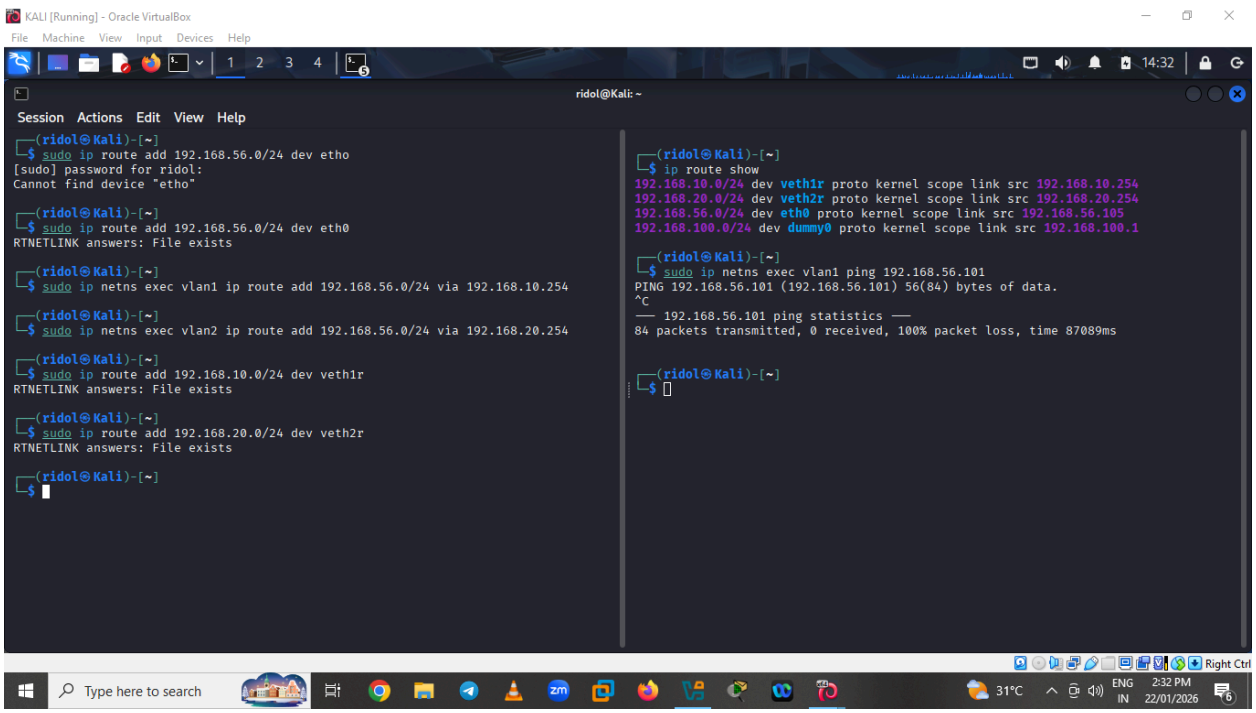

Fig 9: Evidence showing route configuration for owasp on Linux terminal

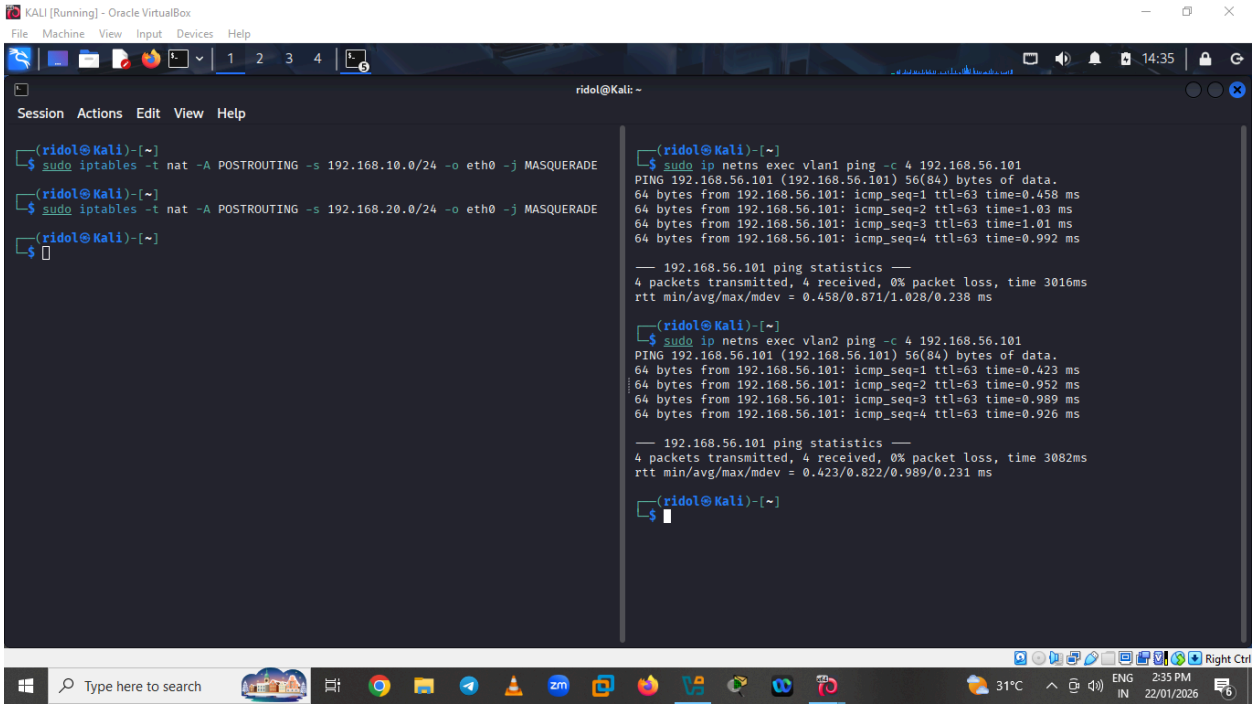

Fig 10: Evidence showing successful ping to owasp from the vlans on Linux terminal

3.4 Connectivity Testing with Ping and Traceroute

Ping tests:

Purpose: Verify all routes and VLAN connectivity.

- **Command** : **`sudo ip netns exec vlan1 ping -c 4 192.168.20.1`** (VLAN2 ping)
- **Command** : **`sudo ip netns exec vlan1 ping -c 4 192.168.56.101`** (OWASP ping)
- **Command** : **`sudo ip netns exec vlan2 ping -c 4 192.168.10.1`** (VLAN1 ping)
- **Command** : **`sudo ip netns exec vlan2 ping -c 4 192.168.56.101`** (OWASP ping)
- **Command** : **`ping -c 4 192.168.56.101`** (OWASP ping)

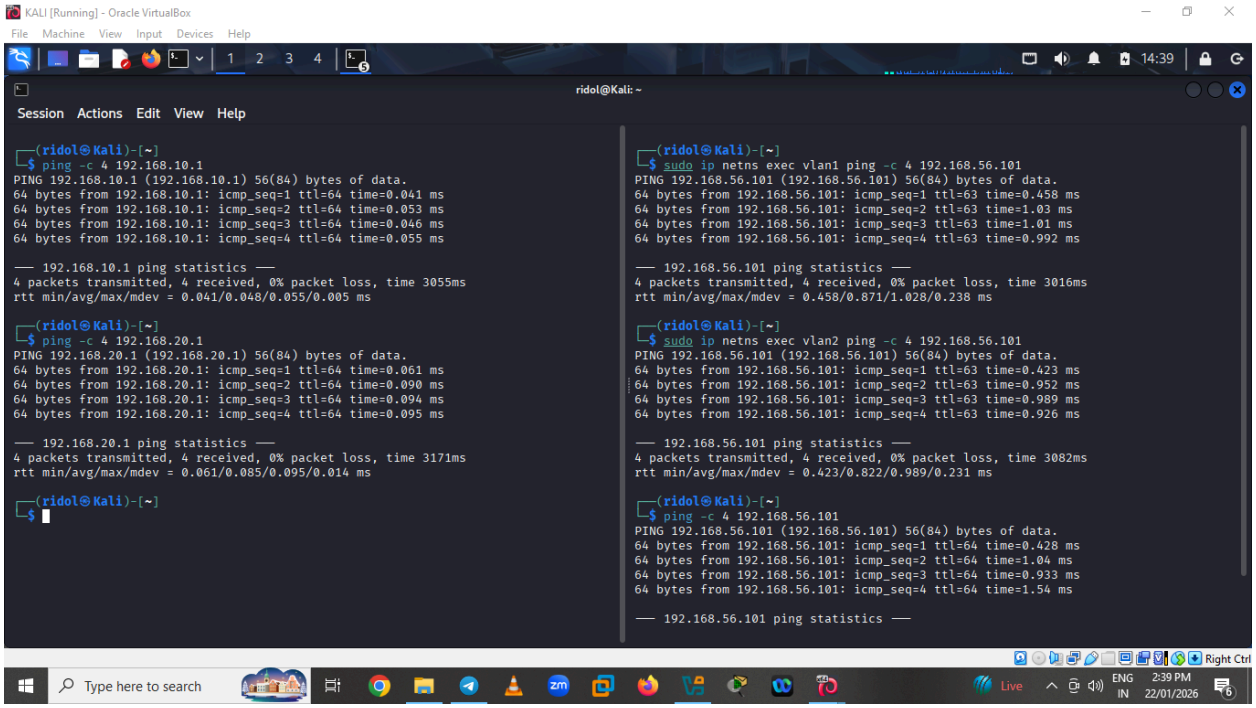

Fig 11: Evidence showing successful pings for the entire inter-network

Traceroute tests:

Purpose: See the path packets take (VLAN - router - OWASP).

- **Command** : **`sudo ip netns exec vlan1 traceroute 192.168.56.101`**
- **Command** : **`sudo ip netns exec vlan2 traceroute 192.168.56.101`**

3.5 iptables Firewall Rules

Flush old rules & enable IP forwarding:

Purpose: Clean start and allow forwarding then control traffic between VLANs and OWASP.

- **Command : `sudo iptables -F`**
- **Command : `sudo iptables -t nat -F`**
- **Command : `sudo iptables -X`**
- **Command : `sudo iptables -P FORWARD ACCEPT`**
- **Command : `sudo sysctl -w net.ipv4.ip_forward=1`**
- **Command : `Sudo cat  /proc/sys/net/ipv4/ip_forward`**

Allow traffic flow between VLAN1 and VLAN2

- **Command : sudo iptables -A FORWARD -s 192.168.10.0/24 -d 192.168.20.0/24 -j ACCEPT**
- **Command : sudo iptables -A FORWARD -s 192.168.20.0/24 -d 192.168.10.0/24 -j ACCEPT**

Block VLAN1 traffic to OWASP VM, allow VLAN2 traffic to OWASP VM:

- **Command : sudo iptables -A FORWARD -s 192.168.10.0/24 -d 192.168.56.101 -j DROP**
- **Command : sudo iptables -A FORWARD -s 192.168.20.0/24 -d 192.168.56.101 -j ACCEPT**

Allow established/related traffic:

- **Command : sudo iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT**

NAT for VLAN traffic to OWASP:

- **Command : sudo iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -o eth0 -j MASQUERADE**
- **Command : sudo iptables -t nat -A POSTROUTING -s 192.168.20.0/24 -o eth0 -j MASQUERADE**

Test firewall behavior:

- **Command : sudo ip netns exec vlan1 ping -c 4 192.168.56.101** (failed as expected)
- **Command : sudo ip netns exec vlan2 ping -c 4 192.168.56.101 (** succeeded as expected)
- **Command : sudo ip netns exec vlan1 ping -c 4 192.168.20.1**

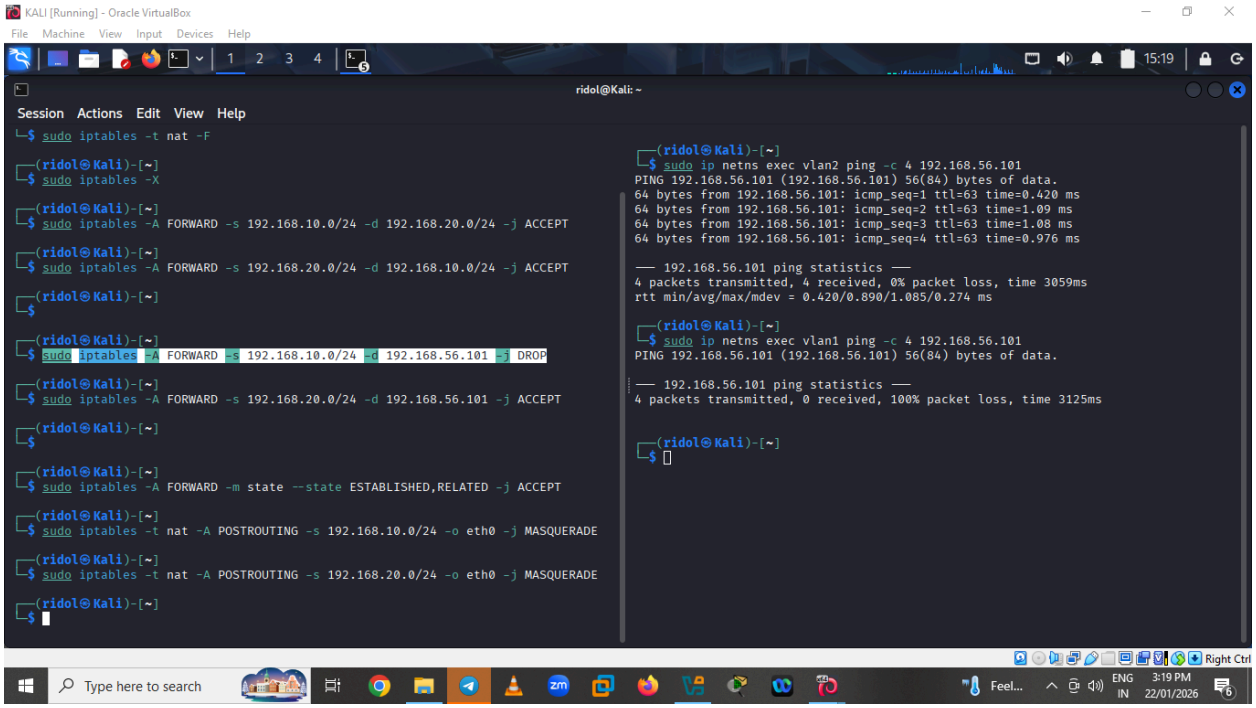

Fig 12: Evidence showing successful firewall configuration on Linux terminal

3.6 Monitoring Traffic with tcpdump

Monitor VLAN interfaces:

- **Command : `sudo ip netns exec vlan1 tcpdump -i veth1h`**
- **Command : `sudo ip netns exec vlan2 tcpdump -i veth2h`**

Monitor Kali router interfaces:

- **Command : `sudo tcpdump -i veth1r`**
- **Command : `sudo tcpdump -i veth2r`**
- **Command : `sudo tcpdump -i eth0 src or dst 192.168.56.101`**

Monitor only ICMP (ping) traffic:

Purpose: Observe packets in real time, confirm NAT/firewall effects, and learn packet flows.

- **Command : `sudo ip netns exec vlan1 tcpdump -i veth1h icmp`**

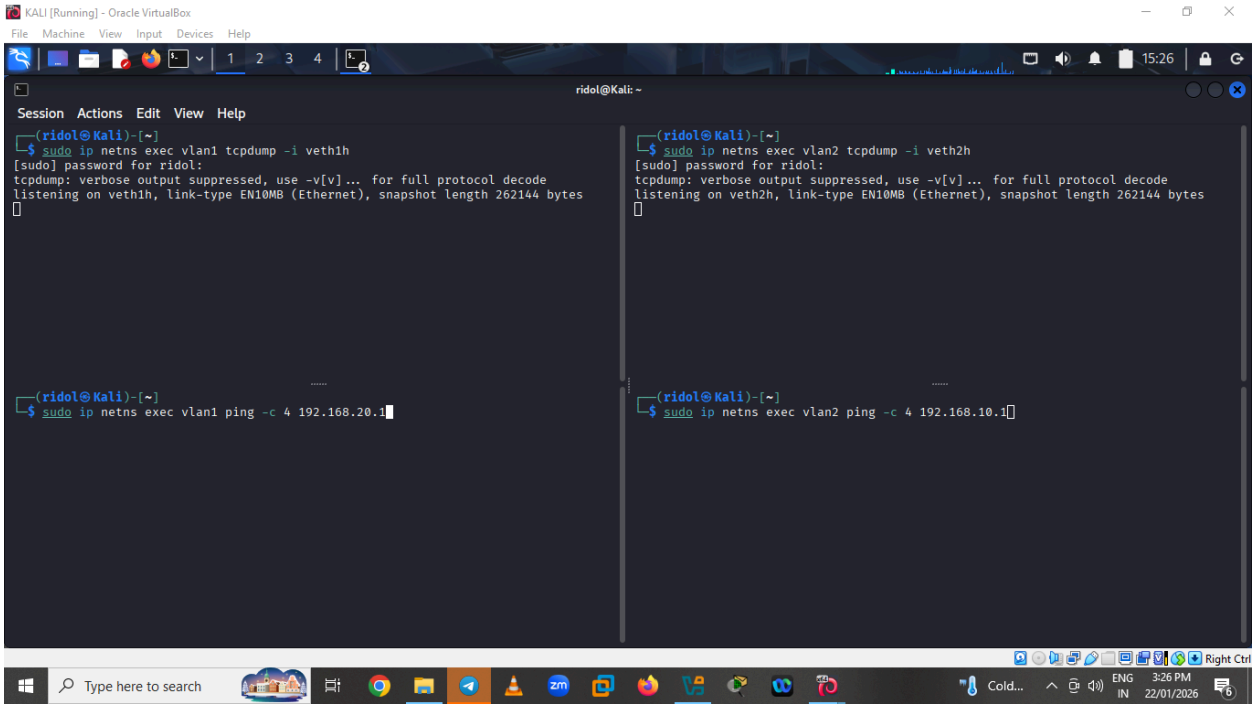

Fig 13: Evidence showing traffic monitoring for vlans communication on Linux terminal

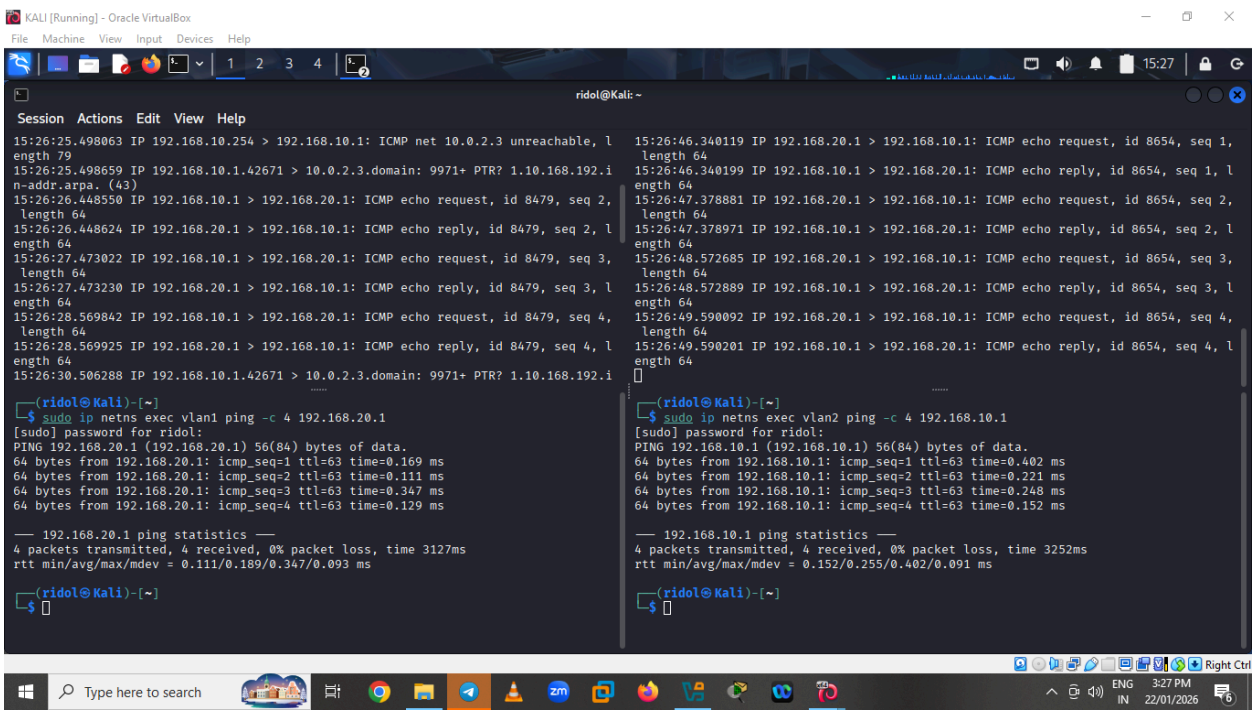

Fig 14: Evidence showing traffic flow for vlans communication on Linux terminal

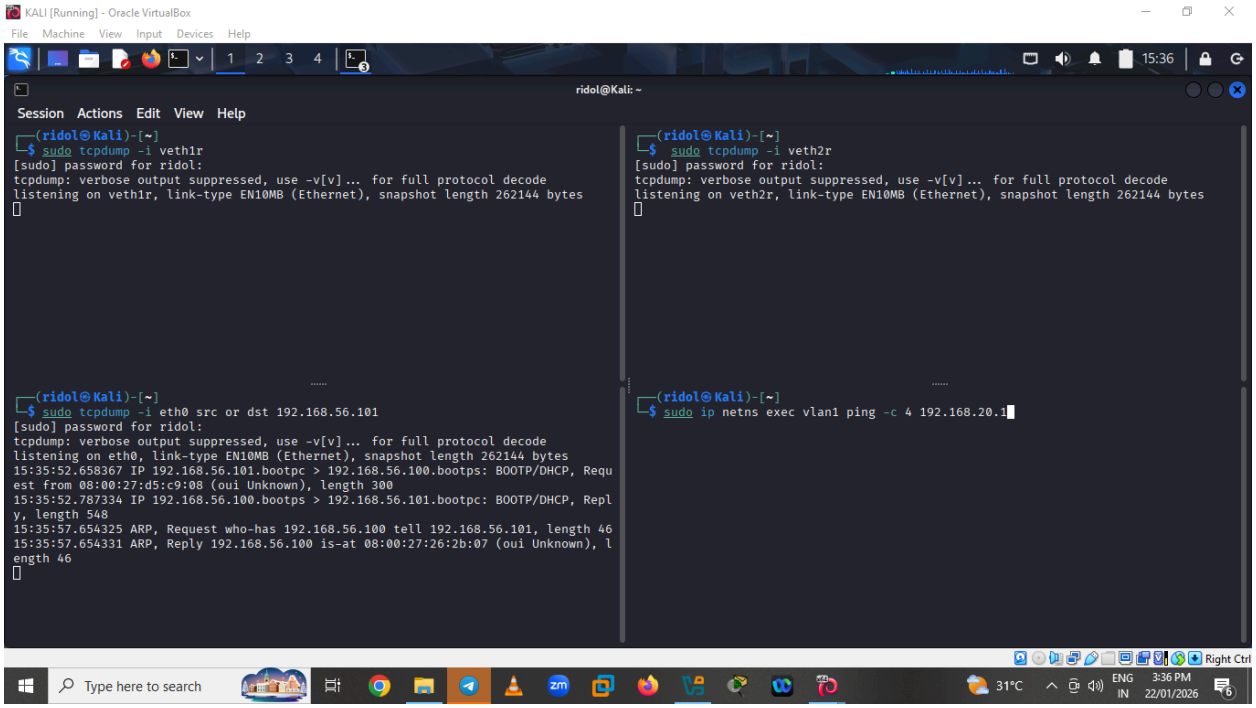

Fig 15: Evidence showing network route monitoring for vlans - owasp communication on Linux terminal

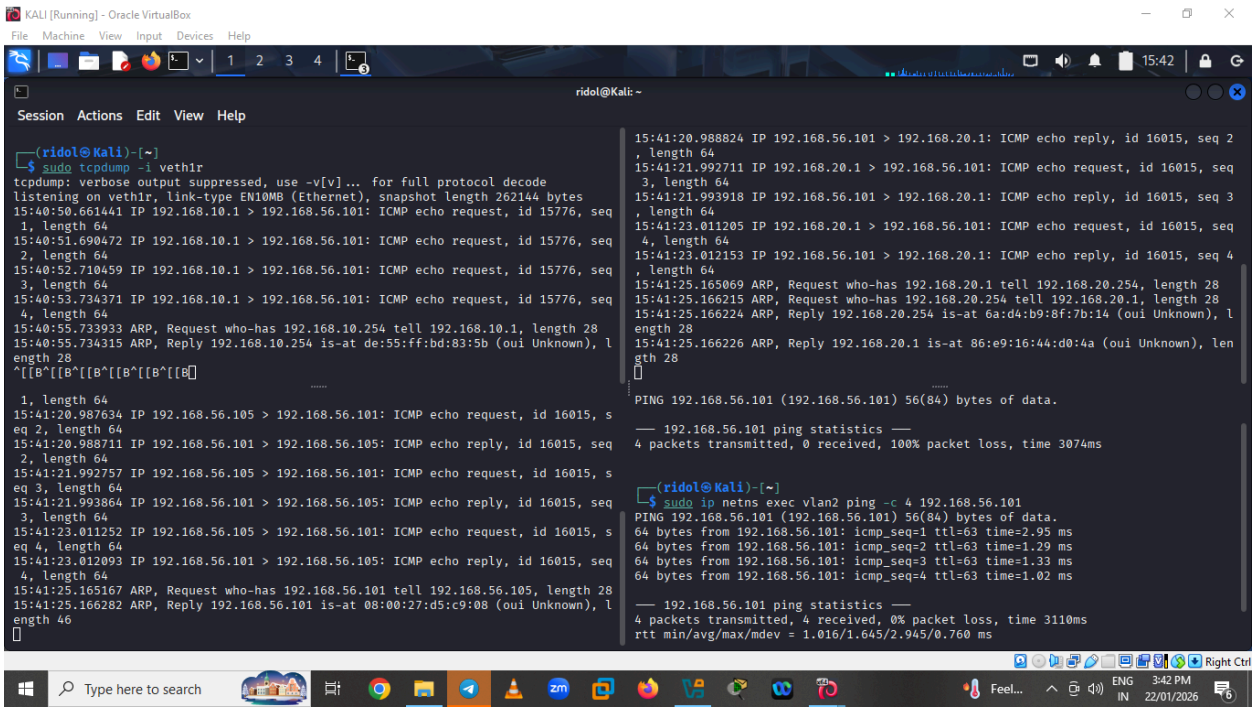

Fig 16: Evidence showing network traffic for vlans - owasp communication on Linux terminal

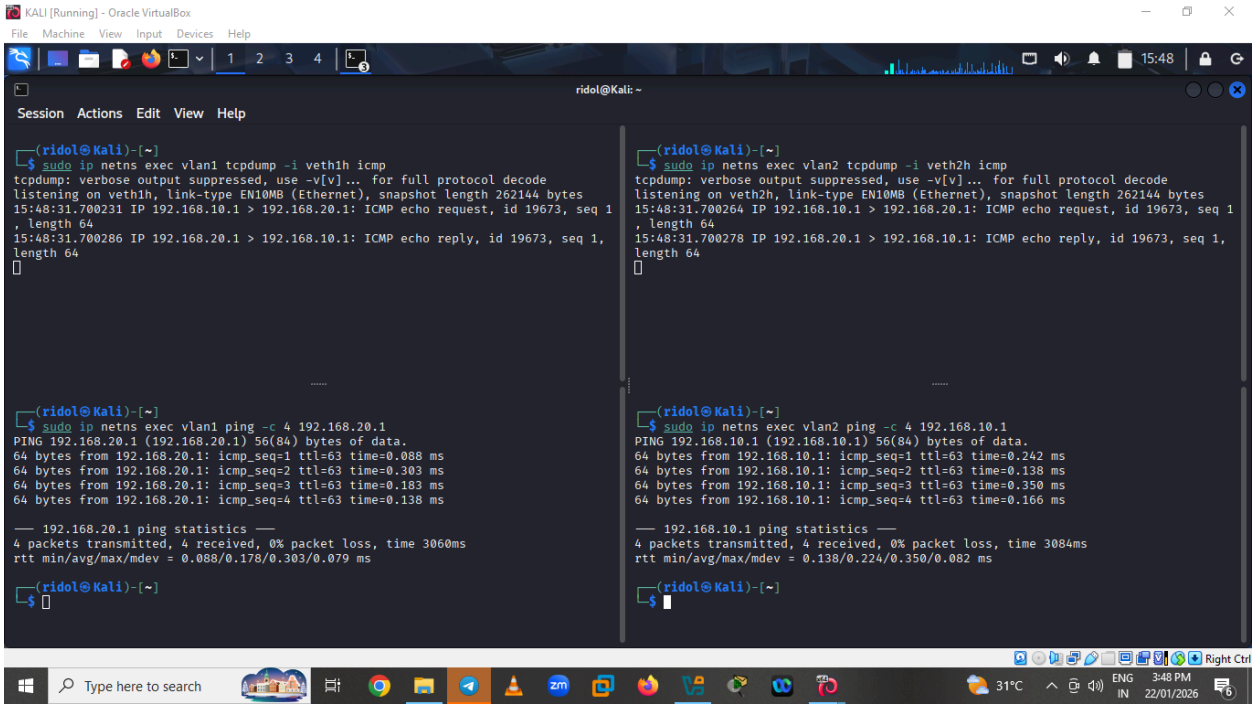

Fig 17: Evidence showing icmp filter traffic monitoring for vlans communication on Linux terminal

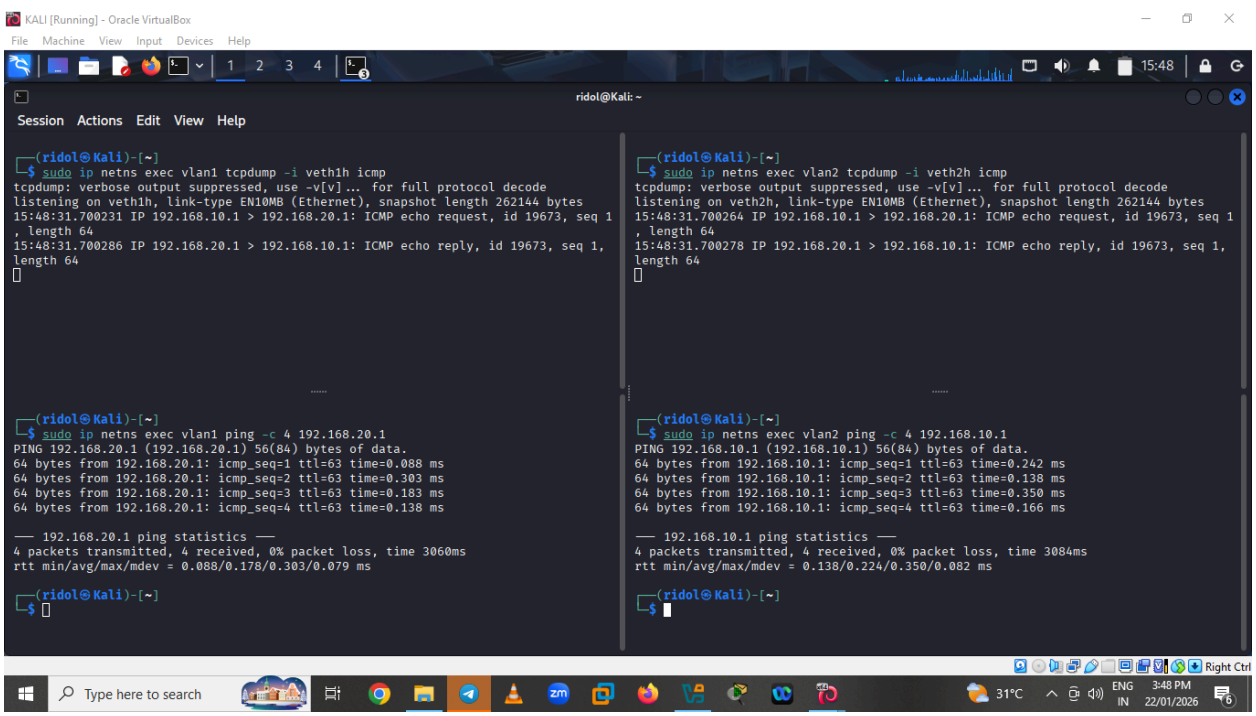

Fig 18: Evidence showing icmp traffic for vlans communication on Linux terminal

4.0 Results and Analysis

The virtual LAN interface acts as a traffic segmentation mechanism to control and propagate traffic from different users or same users for effective communication management.
The successful communication of vlan1 and vlan2 tends to be the backbone of further configuration in the lab. The blockage of packets flow from 192.168.10.0/24 network to owasp 192.168.56.101 indicates a successful firewall configuration.

It's important to note that while configuring the firewall rules the destination ip 192.168.56.101 needs to be defined strictly rather than the whole network 182.168.56.0/24 to avoid all packets from 192.168.56.0 to be dropped instead of just 192.168.56.101 especially when defining a specific target and not the whole network.

5.0 Conclusion and Recommendation

The traffic route from vlans to owasp network was observed to change from 30 hops of undefined route to only the configured route involving a predefined default route and a gateway indicating a successful routing configuration.
VLANs settings and configuration on a network improve network security and traffic management .

6.0 Challenges and Resolution

Challenges encountered include destinations unreachable for vlan to vlan pings which was resolved by thorough troubleshooting process of the route and packet forwarding configuration.